

Session-9 : Stream & Pipeline Processing-

8a- Group Anagrams :

Program:

```
class Solution {  
    public List<List<String>> groupAnagrams(String[] strs) {  
        Map<String, List<String>> map = new HashMap<>();  
        for (String s : strs) {  
            char[] chars = s.toCharArray();  
            Arrays.sort(chars);  
            String key = new String(chars);  
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);  
        }  
        return new ArrayList<>(map.values());  
    }  
}
```

Output:

The screenshot displays the LeetCode submission interface for the 'Group Anagrams' problem. The top section shows the problem description and a submission status of 'Accepted' with 129/129 test cases passed. The submission was made by 'Praveen_1437' on September 15, 2026, at 11:25. A banner for 'Unlock the Full LeetCode Experience' is visible, offering a ₹583.25 discount. The 'Runtime' section shows a runtime of 6ms, which is 98.98% faster than other submissions, and a memory usage of 48.92 MB, which is 99.28% better. A bar chart shows the distribution of runtimes for other submissions. The 'Code' section displays the Java solution, which is the same as the one provided in the text. The 'Test Result' section shows three test cases passed, with a total runtime of 1ms. The input for Case 1 is ["eat","tea","tan","ate","nat","bat"], and the output is [{"eat","tea","ate"}, {"bat"}, {"tan","nat"}].

```
class Solution {  
    public List<List<String>> groupAnagrams(String[] strs) {  
        Map<String, List<String>> map = new HashMap<>();  
        for (String s : strs) {  
            char[] chars = s.toCharArray();  
            Arrays.sort(chars);  
            String key = new String(chars);  
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);  
        }  
        return new ArrayList<>(map.values());  
    }  
}
```

Accepted Runtime: 1 ms

Case 1 Case 2 Case 3

Input

strs =

["eat","tea","tan","ate","nat","bat"]

Output

[["eat","tea","ate"],["bat"],["tan","nat"]]

Expected

[["bat"],["nat","tan"],["ate","eat","tea"]]

9b-Top K Frequent Elements:

Program:

```
class Solution {  
  
    public int[] topKFrequent(int[] nums, int k) {  
  
        Map<Integer, Integer> freqMap = new HashMap<>();  
  
        for (int num : nums) {  
  
            freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);  
  
        }  
  
        PriorityQueue<Map.Entry<Integer, Integer>> pq =  
  
            new PriorityQueue<>((a, b) -> b.getValue() - a.getValue());  
  
        pq.addAll(freqMap.entrySet());  
  
        int[] result = new int[k];  
  
        for (int i = 0; i < k; i++) {  
  
            result[i] = pq.poll().getKey();  
  
        }  
  
        return result;  
  
    }  
  
}
```

Output:

The screenshot shows the LeetCode interface for problem 347. The left pane contains the problem description: "Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in **any order**." It includes three examples with input and output, and constraints: `1 <= nums.length <= 105`, `-104 <= nums[i] <= 104`, and `k` is in the range `[1, the number of unique elements in the array]`.

The right pane shows a code editor with the following Java solution:

```
1 class Solution {  
2     public int[] topKFrequent(int[] nums, int k) {  
3         Map<Integer, Integer> freqMap = new HashMap<>();  
4         for (int num : nums) {  
5             freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);  
6         }  
7         PriorityQueue<Map.Entry<Integer, Integer>> pq =  
8             new PriorityQueue<>((a, b) -> b.getValue() - a.getValue());  
9         pq.addAll(freqMap.entrySet());
```

Below the code editor, the "Test Result" section shows "Accepted" with a runtime of 1 ms. It lists three test cases, all of which are passed. The input for Case 1 is `nums = [1, 1, 1, 2, 2, 3]` and `k = 2`, with the expected output `[1, 2]`.